

Project Report

Handwritten Character Recognition

Kousar Kousar

ID # 22301044

Course Title: Neural Networks

Summary:

This project focuses on handwritten character recognition using a multilayer neural network structure and the MNIST dataset. The key findings include the impact of different hidden layer sizes, activation functions, learning coefficients, and the introduction of L2 weight regularization on the recognition performance. The report details the steps taken, the methodologies applied, and the results obtained from various configurations.

Introduction:

Background and Context:

Handwritten character recognition is a fundamental task in machine learning with applications in various domains. The MNIST dataset, consisting of 70,000 28x28 pixel grayscale images of handwritten digits, serves as the foundation for this project.

Problem Statement and Objectives:

The project aims to explore the impact of different neural network configurations on handwritten character recognition. Specifically, the focus is on hidden layer sizes, activation functions, learning coefficients, and L2 weight regularization.

Literature Review:

An overview of existing methodologies in handwritten character recognition, emphasizing the significance of neural network structures and their components.

Methodology:

Dataset: The MNIST dataset, comprising 60,000 training samples and 10,000 test samples, is utilized. The images are flattened to vectors of length 784, normalized for faster training.

Advantage of Flattening and Normalization: The combination of flattening and normalization prepares the input data for the neural network. The flattened vector provides a one-dimensional representation of the 28x28 image, and normalization ensures that the pixel values are within a consistent and manageable range for efficient training.

These operations are typically performed as a preprocessing step before feeding the data into the neural network for training. The resulting normalized vector serves as the input feature for the subsequent layers of the neural network.

Integrating these steps into the data preprocessing phase ensures that the neural network can effectively learn from the handwritten character images.

Model Architecture: A two-layer structure is employed with a hidden layer having three different sizes ($N = 300, 500, 1000$). Two different non-linear activation functions are considered: tanh for Case 1 and ReLU for the hidden layer and sigmoid for the output layer in Case 2.

The use of non-linear activation functions (tanh for Case 1, ReLU for the hidden layer in Case 2, and sigmoid for the output layer in Case 2) introduces non-linearity into the model. This non-linearity is crucial for enabling the neural network to learn and extract complex hierarchical features from the input data.

Output Pre assignment: Output neurons are preassigned to digits, and desired outputs are defined based on the chosen non-linearities.

Error Function: Mean squares error is used as the error function to be minimized during backpropagation.

Mean Squared Error (MSE) is a continuous and differentiable function. This property is crucial for the backpropagation algorithm, which relies on the computation of gradients to update the neural network's weights.

Learning Coefficient: Three different learning coefficients (0.01, 0.05, 0.09) are considered.

The careful selection of learning coefficients is a critical aspect of optimizing the training process for neural networks in character recognition tasks.

Training: Backpropagation is executed for all combinations of hidden layer sizes, activation functions, and learning coefficients. A stopping criterion is set based on the number of epochs= 50 and the convergence of training set error.

Mini Batch Training: The best-performing configuration is identified based on the test set error/classification accuracy. Mini-batch training is then applied with three different mini-batch sizes (10, 50, 100) for the selected configuration.

L2 Regularization: L2 weight regularization is introduced for the best-performing configuration from the mini-batch training.

L2 regularization, also known as weight decay, introduces a penalty term to the loss function that discourages the model from assigning excessively large weights to parameters. This helps prevent overfitting by penalizing overly complex models.

The regularization cost is evaluated with two different lambda values (0.001, 0.01).

Results Display:

The code prints the accuracy for each configuration, displaying the results in a structured format.

Results:

FOR 50 EPPOCHS			
No. of Neurons	Learning Rate	Cases	Accuracy
300	0.01	Case 1	96.67
300	0.01	Case 2	93.04
500	0.01	Case 1	96.38
500	0.01	Case 2	93.26
1000	0.01	Case 1	95.8
1000	0.01	Case 2	93.25
300	0.05	Case 1	97.96
300	0.05	Case 2	96.95
500	0.05	Case 1	97.92
500	0.05	Case 2	97.06
1000	0.05	Case 1	97.78
1000	0.05	Case 2	97.11
300	0.09	Case 1	97.97
300	0.09	Case 2	97.66
500	0.09	Case 1	98.02
500	0.09	Case 2	97.66
1000	0.09	Case 1	98.05
1000	0.09	Case 2	97.75

```
urons': 300, 'Learning Rate': 0.01, 'Activation': 'case1', 'Epoch': 50, 'Test Accuracy': array(96.67)}
urons': 300, 'Learning Rate': 0.01, 'Activation': 'case2', 'Epoch': 50, 'Test Accuracy': array(93.04)}
urons': 500, 'Learning Rate': 0.01, 'Activation': 'case1', 'Epoch': 50, 'Test Accuracy': array(96.38)}
urons': 500, 'Learning Rate': 0.01, 'Activation': 'case2', 'Epoch': 50, 'Test Accuracy': array(93.26)}
urons': 1000, 'Learning Rate': 0.01, 'Activation': 'case1', 'Epoch': 50, 'Test Accuracy': array(95.8)}
urons': 1000, 'Learning Rate': 0.01, 'Activation': 'case2', 'Epoch': 50, 'Test Accuracy': array(93.25)}
urons': 300, 'Learning Rate': 0.05, 'Activation': 'case1', 'Epoch': 50, 'Test Accuracy': array(97.96)}
urons': 300, 'Learning Rate': 0.05, 'Activation': 'case2', 'Epoch': 50, 'Test Accuracy': array(96.95)}
urons': 500, 'Learning Rate': 0.05, 'Activation': 'case1', 'Epoch': 50, 'Test Accuracy': array(97.92)}
urons': 500, 'Learning Rate': 0.05, 'Activation': 'case2', 'Epoch': 50, 'Test Accuracy': array(97.06)}
urons': 1000, 'Learning Rate': 0.05, 'Activation': 'case1', 'Epoch': 50, 'Test Accuracy': array(97.78)}
urons': 1000, 'Learning Rate': 0.05, 'Activation': 'case2', 'Epoch': 50, 'Test Accuracy': array(97.11)}
urons': 300, 'Learning Rate': 0.09, 'Activation': 'case1', 'Epoch': 50, 'Test Accuracy': array(97.97)}
urons': 300, 'Learning Rate': 0.09, 'Activation': 'case2', 'Epoch': 50, 'Test Accuracy': array(97.66)}
urons': 500, 'Learning Rate': 0.09, 'Activation': 'case1', 'Epoch': 50, 'Test Accuracy': array(98.02)}
urons': 500, 'Learning Rate': 0.09, 'Activation': 'case2', 'Epoch': 50, 'Test Accuracy': array(97.66)}
urons': 1000, 'Learning Rate': 0.09, 'Activation': 'case1', 'Epoch': 50, 'Test Accuracy': array(98.05)}
urons': 1000, 'Learning Rate': 0.09, 'Activation': 'case2', 'Epoch': 50, 'Test Accuracy': array(97.75)}
```

Results Summary:

Case 1 (tanh-tanh):

Best Configuration: N=1000, Hidden Activation=tanh, Output Activation=tanh, Accuracy= 98.05%

Observations:

The choice of activation functions significantly affects the performance. tanh-tanh configuration achieves much better higher efficiency which is 80.94%.

The model achieves consistently high accuracy across various configurations. 'Case 1' (tanh) consistently outperforms 'Case 2' (sigmoid) in terms of accuracy for all configurations.

As the number of neurons increases, the accuracy generally improves.

Higher learning rates (0.05 and 0.09) contribute to improved accuracy, but there might be diminishing returns at extremely high learning rates.

Overall, the model is performing well, with accuracies reaching above 97% in most configurations.

Recommendations:

The code can be extended to handle more complex architectures or different datasets for broader experimentation.

Saving the best model weights and biases for each case can be useful for future use or analysis.

Conclusion:

This project serves as a comprehensive exploration of hyperparameter configurations for a neural network on the MNIST dataset, highlighting the importance of careful tuning for achieving optimal performance.